

C'est quoi ORM ?

Un ORM (Object-Relational Mapping) est une technique qui permet de manipuler une base de données avec des objets au lieu d'écrire du SQL.

Comprendre avec une analogie

Base de données	Code PHP
Table	Classe
Colonne	Attribut
Ligne	Objet

Select * from users ↔ **User::all()**

1. Récupération des données (READ)

- **Toutes les données**
`User::all();`
- **Trouver par ID**
`User::find(1);`
`User::findOrFail(1);`
- **Premier résultat**
`User::first();`
- **Conditions (WHERE)**
`User::where('age', '>', 18)->get();`
- **Plusieurs conditions**
`User::where('age', '>', 18)`
`->where('status', 'active')`
`->get();`
- **OR condition**
`User::where('age', '<', 18)`
`->orWhere('role', 'admin')`
`->get();`
- **whereIn**
`User::whereIn('id', [1,2,3])->get();`
- **whereNull**
`User::whereNull('email_verified_at')->get();`
- **whereLike**
`User::where('name', 'like', '%ali%')->get();`

2. Tri et limitation

- **OrderBy**
`User::orderBy('name', 'asc')->get()`
- **Limit**
`User::take(5)->get();`
- **Pagination**
`User::paginate(10)->get()`

3. Agrégation

- `User::count();`
- `User::max('age');`
- `User::min('age');`
- `User::avg('age');`
- `User::sum('salary');`

4. Insertion (CREATE)

- **Create**
`User::create([
 'name' => 'Ali',
 'email' => 'ali@gmail.com'
]);`
- **Save**
`$user = new User();
$user->name = "Ali";
$user->email = "ali@gmail.com";
$user->save();`

5. Mise à jour (UPDATE)

- **Update simple**
`$user = User::find(1);`
`$user->name = "Ahmed";`
`$user->save();`
- **Update direct**
`User::where('id', 1)->update([`
 `'name' => 'Ahmed'`
`]);`

6. Suppression (DELETE)

```
User::find(1)->delete();  
User::where('age', '<', 18)->delete();
```

7. Relations

- **One to Many**
`$user->posts;`
- **Charger relation**
`User::with('posts')->get();`
- **Charger relation**
`User::whereHas('posts', function($q)`
 `{`
 `$q->where('title', 'like', '%Laravel%');`
 `})->get();`

8. Les Jointures (JOIN)

- **INNER JOIN**

```
$users = DB::table('users')  
    ->join('posts', 'users.id', '=',  
          'posts.user_id')  
    ->select('users.name',  
          'posts.title')  
    ->get();
```

- **LEFT JOIN**

```
$users = DB::table('users')  
    ->leftJoin('posts', 'users.id',  
             '=', 'posts.user_id')  
    ->get();
```

```
use Illuminate\Support\Facades\DB;
```

- **RIGHT JOIN**

```
$users = DB::table('users')  
    ->rightJoin('posts', 'users.id',  
              '=', 'posts.user_id')  
    ->get();
```

- **CROSS JOIN**

```
$users = DB::table('users')  
    ->crossJoin('roles')  
    ->get();
```

9. UNION

- **CROSS JOIN**

```
$results = $q1->union($q2)->get();  
$results = $q1->unionAll($q2)->get();
```

6. Base de Données & Eloquent ORM

Application

1. Récupérer tous les utilisateurs
2. Récupérer l'utilisateur avec id = 5
3. Récupérer les utilisateurs dont le nom contient "Ali"
4. Récupérer les 10 premiers utilisateurs
5. Récupérer les posts dont le titre contient "Laravel"
6. Récupérer les posts créés par l'utilisateur id = 3
7. Récupérer les commentaires du post id = 2
8. Récupérer tous les posts avec leurs commentaires
9. Récupérer tous les utilisateurs avec leurs posts
10. Récupérer les commentaires avec les informations du post associé
11. Faire une jointure entre users et posts pour afficher : nom utilisateur et titre du post
12. Faire une LEFT JOIN entre posts et comments
13. Compter le nombre total de posts
14. Compter le nombre de commentaires pour chaque post

